



UDB

Universal Data Broker is the typed control point for application data. Teams declare the domain once in protobuf, enforce tenant/project/security centrally, route to many backends, and call the result from any supported SDK.

● **DECLARE ONCE**

● **ENFORCE CENTRALLY**

● **ROUTE TO 18 BACKENDS**

● **CALL FROM 6 SDKS**

● **EXTEND WITH UDB PLUS**

CORE

One DataBroker API for records, objects, vectors, cache, CDC, transactions, catalog, migration, and health.

CONTROL

Native authn, authz, IdP, API keys, tenants, storage, assets, WebRTC, policy, analytics, and notifications.

SECURITY

Tenant/project context, scopes, purpose, service identity, redaction metadata, audit, and fail-closed checks.

PLUS

Paid protocol packs for HL7v2, FHIR, MQTT, OPC UA, and future domain protocols.

UDB PRODUCT MODEL

Project proto

Application entities, services, storage annotations, endpoint security, field security, SDK surfaces, and events.

UDB Core

Descriptor manifest, DataBroker runtime, native services, routing, migrations, CDC, health, and CLI.

Backends

SQL, vector, object, cache, document, graph, search, analytical, and column stores behind one contract.

SDKs

Go, Python, TypeScript/Node, Java, C#, PHP/Laravel, generated clients, workflow helpers, and metadata rules.

UDB Plus

Optional paid protocol packs add dictionaries, codecs, transports, validators, mappings, and evidence.

UDB turns fragmented application infrastructure into one governed broker foundation.

Growing products accumulate the same problems: data in many stores, tenant rules repeated in services, auth logic copied at the edges, SDK drift, schema drift, migrations, audit gaps, and integration scripts. UDB moves the contract into protobuf descriptors and makes the broker enforce that contract on every request.

01

01**Declare once**

Project protos define entities, tables, columns, stores, endpoint security, field security, SDK surfaces, event contracts, and docs.

02**Enforce centrally**

Every non-health request carries tenant, project, purpose, scopes, service identity, user identity, correlation id, and catalog version.

03**Route honestly**

The runtime resolves the active catalog, verifies operation capability, checks backend health, lowers through neutral IR, and refuses unsupported paths.

04**Operate as a platform**

Health, compatibility matrix, migrations, CDC, DLQ, replay, audit, native services, SDKs, and CLI become shared infrastructure.

WITHOUT UDB**Every product becomes its own data platform.**

Teams rebuild API layers, auth filters, SDK wrappers, backend routing, migration scripts, audit tables, integration jobs, and operational checks. Each new customer or protocol adds more custom glue.

WITH UDB**The descriptor is the contract; the broker is the enforcement point.**

Teams model the domain once, choose backends and optional protocol packs, and let UDB own routing, authorization, generated clients, migrations, CDC, audit, health, and operational evidence.

Sales message: UDB is not just CRUD over gRPC. It is the foundation layer for configurable, secure, multi-backend, SDK-ready products.

What UDB gives a product team.

UDB ships a public data plane, an internal native control plane, descriptor-driven generation, multi-backend routing, operational events, and SDKs. The same foundation can support healthcare, industrial, government, SaaS, and internal platform products because the broker contract is stable while project schemas vary.

77

DataBroker RPCs

Records, objects, vectors, cache, documents, graphs, analytics, catalog, migrations, transactions, CDC, DLQ, and admin health.

188

Control-plane RPCs

15 native services for authn, authz, API keys, IdP, tenants, notifications, analytics, storage, assets, WebRTC, and policy.

18

Backend kinds

Postgres, MySQL, SQLite, SQL Server, ClickHouse, Redis, Memcached, Qdrant, Weaviate, Pinecone, MinIO, S3, Azure Blob, GCS, MongoDB, Elasticsearch, Neo4j, Cassandra.

6

SDK families

Go, Python, TypeScript/Node, Java, C#, and PHP/Laravel clients share metadata, credentials, generated protos, and workflow helpers.

UDB CAPABILITY	WHAT IT MEANS	WHY BUYERS CARE
Proto source of truth	Descriptors and annotations define storage, security, SDK metadata, events, and generated docs.	Fewer hidden contracts and less drift between API, database, SDK, and runtime behavior.
Backend-neutral broker	Requests lower into neutral operations and compile to backend-specific execution paths.	Products can use the right store without making every app team learn every backend.
Native control plane	Identity, authorization, tenants, storage metadata, assets, WebRTC, notifications, analytics, and policy run as platform services.	Teams stop rebuilding enterprise plumbing in every application.
Operational contract	Doctor, health-check, compat-matrix, migration, drift, CDC, DLQ, replay, and audit are part of the product surface.	UDB is supportable infrastructure, not a thin data-access library.

How UDB modernizes legacy and fragmented products without forcing a rewrite.

UDB does not require every existing system to disappear. It gives legacy databases, scattered services, files, vectors, caches, documents, graphs, analytics stores, and native services a typed broker layer that new applications and SDKs can consume consistently.

LEGACY OR FRAGMENTED REALITY	UDB MODERNIZATION PATH	RESULT
Existing SQL databases and old app tables	Describe entities and table/column mappings in proto; generate catalog, SQL, migrations, SDK metadata, and runtime routing.	Existing data gains a typed, governed API without forcing an immediate database rewrite.
Different stores for files, search, vectors, cache, documents, graphs, and analytics	Use UDB backend kinds and neutral IR to route operation families to the correct configured store.	One application contract can span many backend technologies.
Repeated auth and tenant checks in every service	Put tenant, project, purpose, scopes, user, service identity, and catalog version in request metadata and enforce centrally.	Security posture becomes consistent across reads, writes, native services, and protocol traffic.
Customer-specific integrations and protocol feeds	Keep UDB Core as the foundation; attach UDB Plus protocol packs where a project needs domain integration.	Domain readiness becomes a project capability instead of a one-off integration engine.

Healthcare

Connect legacy HL7v2 feeds, modern FHIR APIs, storage, policy, SDKs, and audit through one broker foundation.

Industrial and IoT

Bring telemetry, device identity, commands, node models, time-series data, and operational evidence into project-owned APIs.

Government and enterprise

Support strict tenant boundaries, access control, identity federation, reporting, long retention, and controlled change.

Everything starts from proto.

UDB uses protobuf descriptors as the source of truth for the application contract. Runtime checks, generated manifests, docs, SQL artifacts, SDKs, and service metadata derive from that descriptor contract instead of hand-maintained lists.

04

1. Model

Project protos define entities, events, services, security, storage annotations, and SDK surfaces.

2. Compile

UDB builds a descriptor manifest, catalog, SQL, SDK manifests, protocol indexes, and generated docs.

3. Generate

UDB emits catalog, backend artifacts, SDK manifests, generated docs, and compatibility metadata from the same contract.

4. Execute

The broker enforces tenant/project context, authorization, routing, admission, backend execution, audit, and events.

5. Operate

Health, drift, replay, DLQ, migration, pack compatibility, and SDK conformance are checked through the same control plane.

Descriptor-driven

- No handwritten API inventory.
- Generated SDKs and docs derive from proto.
- Method, field, table, event, and protocol metadata share one contract.

Tenant/project scoped

- Every call carries tenant, project, purpose, user, scopes, service identity, and correlation id.
- Runtime rejects unresolved scope instead of guessing.
- Pack operations inherit the same security boundary.

Backend-neutral

- Postgres, MySQL, SQLite, SQL Server, ClickHouse, Redis, MongoDB, MinIO/S3, Neo4j, vector stores, and more.
- Protocol data can land in project-owned canonical models.

Enterprise controls are part of the broker foundation.

UDB security is based on explicit request context, descriptor-declared endpoint security, native auth/authz services, backend-aware enforcement, redaction metadata, audit events, and a separate internal control-plane listener.

CONTROL AREA	UDB SUPPORT	BUSINESS VALUE
Request context	Tenant id, project id, purpose, correlation id, scopes, service identity, user id, and client catalog/protocol version.	Every read, write, native-service call, and protocol-pack operation can be bound to a real project and tenant boundary.
Authentication	JWT validation, UDB-issued JWTs, sessions, refresh tokens, API keys, MFA/OTP, devices, WebAuthn, and external identity mapping.	Products get a native identity surface instead of building a separate auth subsystem per app.
Authorization	RBAC, ABAC, ReBAC, relationship tuples, policy bundles, access decision audits, and restricted native access.	Data and protocol operations can be authorized before execution, not after damage is done.
Operations	Audit events, CDC, DLQ, replay, health, compat-matrix, doctor, migration gates, and policy distribution.	UDB is operable infrastructure with evidence, diagnostics, and recovery posture.

15 native services

Authn, authz, API keys, IdP, control distribution, tenant, notification, analytics, storage, asset, and WebRTC services.

Separate listener

The native control plane can be bound internally or placed behind a trusted gateway, separate from public DataBroker traffic.

Fail closed

Unresolved identity, tenant, scope, policy, backend capability, or descriptor state rejects instead of guessing.

UDB makes the broker usable from real application stacks.

SDKs attach metadata, expose DataBroker helpers, provide native-service clients, include workflow facades, and keep each language tied to the same descriptor-derived contract. Operations use the CLI and generated manifests to inspect runtime readiness.

Workflow client

Common flows such as login plus tenant adoption, file upload, file download, authz helpers, entity CRUD, and read-after-write metadata stay short.

Generated clients

Advanced and admin users can call the full descriptor-derived gRPC surface without inventing a second protocol.

Conformance

Cross-language tests check metadata headers, credentials, authz scopes, policy bundles, refresh behavior, and facade routing.

UDB OPERATION	USEFUL COMMAND OR SURFACE	WHY IT MATTERS
Export shared protos	udb proto export --fmt	Applications import UDB annotations instead of copying broker contracts by hand.
Inspect contract	udb catalog proto, udb native manifest, udb native docs	Teams can see what the descriptor actually declares.
Check deployment	udb doctor --human, udb health-check, udb compat-matrix	Operators know which backends and services are mounted and healthy.
Generate clients	udb sdk generate --lang all	SDK surfaces stay tied to proto, descriptor metadata, package version, and protocol version.

UDB Plus adds paid protocol packs on top of the broker foundation.

UDB Core is the required foundation. UDB Plus is the commercial extension layer for projects that need protocol behavior in addition to normal application data: parse, serialize, validate, map, transport, acknowledge, store, audit, and expose protocol workflows through SDKs.

07

UDB CORE

The platform every project needs.

DataBroker, native control plane, proto annotations, backend routing, security, SDKs, migrations, CDC, audit, health, CLI, and operational checks.

UDB PLUS

The protocol layer selected per project.

Protocol packs add dictionaries, source facts, compiled plans, parsers, serializers, validators, transport adapters, mappings, evidence, reports, and SDK protocol indexes.

UDB + HL7v2

Healthcare messaging, MSH version selection, dictionaries, parser/serializer, validation, ACK posture, and MLLP boundary.

UDB + FHIR

FHIR resources, profiles, terminology, bundles, search/query mapping, provenance, and policy-aware resource workflows.

UDB + MQTT

Topics, device identity, payload schemas, QoS posture, retained state, telemetry ingestion, command routing, and projections.

UDB + OPC UA

Namespaces, nodes, datatypes, monitored items, methods, alarms, certificates, secure channels, and industrial evidence trails.

Sales message: UDB sells the foundation. UDB Plus sells domain readiness as a project-selectable capability.

Core platform first, protocol packs second.

The packaging should be simple for a buyer: every customer needs UDB Core. Projects that need healthcare messaging, FHIR APIs, IoT telemetry, or industrial node models attach UDB Plus packs.

08

OFFERING	WHAT IT INCLUDES	BEST FIT	OPERATIONAL RESULT
UDB Core	Proto annotations, DataBroker API, native services, backend routing, tenant/project security, SDK generation, migrations, CDC, audit, health, and CLI.	Applications that need one typed API across many stores and native platform services.	Less backend glue, one descriptor contract, one broker runtime, and one operational model.
UDB Plus	Protocol-pack runtime, pack manifests, dictionaries, validation plans, transport adapters, compiled artifacts, evidence, generated docs, and SDK protocol indexes.	Products that need domain protocols in addition to normal app data.	The project opts into protocol support without becoming a custom integration engine.
UDB + HL7v2	UDB-owned UPF seed source, parser/serializer, ACK facade, dictionary snapshots, version selection, validation, MLLP boundary, compiled plans, SQLite, and reports.	Healthcare interfaces, labs, EHR integration, ADT, ORM, ORU, SIU, MDM, DFT, VXU, and ACK workflows.	Legacy HL7v2 flows become governed, auditable, rebuildable, and SDK-accessible.
UDB + FHIR / MQTT / OPC UA	Pack families use the same architecture: source facts, compiled plans, runtime behavior, transport semantics, mapping, evidence, and SDK exposure.	Modern healthcare APIs, IoT telemetry, edge gateways, SCADA, industrial automation, and government/enterprise integration.	Different protocols share the same UDB security, storage, SDK, and operations foundation.

A UDB Plus pack is more than a parser.

Each pack carries source facts, compiled runtime artifacts, service boundaries, generated client affordances, and evidence needed to operate safely. Parsing matters, but commercial value comes from the full managed protocol lifecycle.

LAYER	SOURCE FACTS	RUNTIME	SERVICES	EVIDENCE
Facts	Versions, messages, resources, segments, datatypes, fields, tables, profiles, nodes, topics, bindings.	Compiled snapshots, lookup tables, validation plans, codec hints, compatibility manifests.	Request/response protos, service facades, generated SDK indexes, project opt-in metadata.	Source provenance, hashes, coverage summaries, diffs, curation reports, audit posture.
Behavior	Version-specific variance and profile-specific constraints.	Parse, serialize, normalize, validate, map, route, acknowledge, subscribe, publish, browse, and command.	Gateway operations, batch jobs, transport endpoints, subscriptions, replay, and health checks.	Issue reports, validation paths, deterministic fingerprints, replayable errors, and generated reports.
User value	Customers do not hand-code protocol dictionaries.	Projects get protocol behavior without embedding a separate engine.	SDK users call a coherent UDB protocol surface.	Operations can explain exactly what happened and why.

Dictionary

Protocol versions, messages, resources, tables, datatypes, topics, nodes, and profile metadata.

Core

Canonical intermediate representation, references, paths, cardinality, vocab bindings, and provenance.

Transport

MLLP, HTTP/FHIR, MQTT broker, OPC UA secure channel, retries, framing, acknowledgments, and sessions.

Validation

Structural checks, datatype checks, vocabulary checks, profile rules, correction reports, and issue severity.

Legacy healthcare messaging, handled as a first-class protocol.

HL7v2 is not a single schema. It is a family of versions, trigger events, message structures, segments, datatypes, escape rules, table bindings, acknowledgments, and transport conventions. UDB + HL7v2 treats those differences as data and runtime contracts.

2.x

Version-aware dictionaries

Compiled snapshots select behavior from MSH-12, including message, segment, datatype, table, and structure metadata.

ER7

Lossless parser

Preserves raw bytes, MSH separator semantics, segment terminators, repetitions, components, subcomponents, null fields, and non-UTF8 diagnostics.

ACK

Messaging facade

Parse, resolve, validate, serialize, and generate application ACKs behind a protocol-ready service boundary.

HL7V2 CAPABILITY	WHAT UDB PROVIDES	CLIENT VALUE
ADT, ORM, ORU, SIU, MDM, DFT, VXU, ACK	Message-family resolution from MSH-9 and MSH-12, with compiled dictionary lookup.	One platform can ingest administrative, order, result, scheduling, document, billing, immunization, and acknowledgment flows.
Segments and datatypes	Required segment checks, field cardinality, datatype references, component metadata, table references, and warning/error severity.	Validation becomes explainable instead of buried inside interface scripts.
Transport boundary	MLLP-ready framing and commit ACK posture can be separated from HL7 application ACK semantics.	Deployments can integrate with interface engines without mixing transport and message semantics.
Seed pack	UDB-owned UPF source, compiled snapshots, SQLite support, reports, deterministic fingerprints, and curation workflow.	Protocol behavior can be rebuilt, audited, and upgraded without private third-party research folders.

FHIR resources connected to storage, policy, and events.

UDB + FHIR uses the same pack architecture for modern healthcare APIs: resources, profiles, extensions, terminology bindings, bundles, search behavior, and provenance artifacts can connect to project-owned data models and SDKs.

11

Resources

Patient, Encounter, Observation, DiagnosticReport, Medication, Procedure, ImagingStudy, Organization, Practitioner, and custom profiles.

Profiles

ImplementationGuide packaging, StructureDefinition constraints, extensions, slices, cardinality, and profile-specific validation.

Terminology

ValueSet and CodeSystem binding, binding strength, code validation, and versioned terminology provenance.

Operations

Bundle ingest, REST facade, search mapping, resource history, subscriptions, provenance, and consent-aware authorization.

FHIR request

REST, bundle, or SDK call enters the protocol facade.

Profile resolve

Resource type, version, profile, and extensions determine validation posture.

Policy check

Tenant, patient, project, purpose, user, and scopes are checked centrally.

Store/project

Resources map to canonical UDB records, object payloads, events, or project services.

Evidence

Validation, provenance, changes, and event history stay queryable.

Device telemetry and command streams with project governance.

UDB + MQTT is for projects where devices, sensors, gateways, and edge services publish data faster than normal CRUD APIs can model cleanly. The pack turns topics and payloads into typed, policy-aware project data.

MQTT AREA	PROTOCOL PACK RESPONSIBILITY	UDB PLATFORM RESPONSIBILITY
Topics	Topic patterns, wildcards, device groups, tenant/project topic namespaces, and route classification.	Catalog mapping, authorization, event contracts, SDK protocol index, and generated documentation.
Payloads	JSON, protobuf, binary, or custom codecs; schema validation; unit metadata; timestamp and quality handling.	Canonical storage, time-series projection, outbox publication, object archival, and replay.
Sessions	Client identity, retained messages, will messages, QoS posture, duplicate handling, and reconnect behavior.	Tenant/device identity, policy decisions, audit, health, and operational metrics.
Commands	Downlink topics, command correlation, acknowledgment mapping, and delivery reports.	RBAC/ABAC/ReBAC checks, idempotency, audit chain, saga coordination, and DLQ.

IoT products

- Device telemetry ingestion.
- State snapshots and event history.
- Command routing with policy checks.

Facilities and meters

- Sensor values, counters, alarms, energy usage, and environment readings.
- Project-ready APIs for dashboards and reports.

Edge gateways

- Bridge industrial protocols into MQTT topics.
- Forward typed data to UDB while keeping source identity.

Industrial node models without losing broker governance.

UDB + OPC UA is designed for industrial and facility systems where data is organized by namespaces, nodes, datatypes, monitored items, methods, alarms, and secure channels.

Browse and import

Namespace discovery, node id references, object/variable/method classification, datatype mapping, and project model suggestions.

Monitor

Subscriptions, monitored items, sampling, queue policy, timestamps, status codes, alarms, and event payloads.

Control

Write requests, method calls, authorization gates, command receipts, replay posture, and audit evidence.

Secure

Certificates, endpoint policy, session identity, tenant/project mapping, and operational health reporting.

The OPC UA pack is the industrial counterpart to FHIR and HL7v2: preserve source identity, compile protocol metadata, expose safe service operations, and project the result into UDB-managed data, events, and SDKs.

INDUSTRIAL SCENARIO	HOW UDB HELPS	WHY IT MATTERS
SCADA dashboards	Expose live node values and alarms through authorized project APIs.	Operations teams get real-time visibility without direct plant-network coupling.
Predictive maintenance	Project machine telemetry into time-series, vector, document, and analytics backends.	Data science and reliability programs can use the same governed broker path.
Command workflows	Gate writes and method calls behind identity, scope, purpose, and audit controls.	Industrial control remains accountable and reviewable.

One facade for parse, validate, transform, transport, and store.

Protocol packs should feel native to SDK users. The service layer presents high-level operations while the runtime retains lossless artifacts, validation details, acknowledgments, mappings, and transport evidence.

14

Parser and serializer

- Lossless raw-byte capture.
- Canonical serialization.
- Version and delimiter awareness.
- Structured path access.

Validation and profile

- Structural validation.
- Datatype and vocabulary binding.
- Profile rules and issue severity.
- Corrections as reports, not mutation of parsed facts.

Transport and messaging

- MLLP and application ACK separation.
- FHIR HTTP/bundle behavior.
- MQTT session/topic/QoS mapping.
- OPC UA secure channel and subscriptions.

Mapping

Protocol paths map into project entities, events, services, and evidence objects without deleting source provenance.

Storage

Records, object payloads, search indexes, graphs, vectors, analytics, and time-series projections can share UDB routing.

SDKs

Generated clients expose project data plus protocol operations in Go, Python, TypeScript, Java, C#, and PHP/Laravel.

Protocol traffic receives the same controls as application data.

Healthcare and industrial protocols often arrive through side channels. UDB Plus keeps those channels inside the same tenant/project, authorization, audit, and evidence model as normal application APIs.

CONTROL AREA	UDB SUPPORT	PROTOCOL-PACK BENEFIT
Identity and tenant scope	Verified request context, tenant id, project id, user id, purpose, scopes, service identity, and correlation id.	HL7 feeds, FHIR calls, MQTT clients, and OPC UA sessions can be bound to real project boundaries.
Authorization	RBAC, ABAC, ReBAC, policy bundles, native access, and fail-closed checks.	Protocol writes, commands, subscriptions, and transformations can be authorized before execution.
Audit and evidence	Admin audit, event envelopes, provenance, hash chain support, validation reports, and deterministic fingerprints.	Every parse, validation, ACK, mapping, and command can explain what source data produced what result.
Reliability	Outbox, CDC, DLQ, replay controls, saga coordination, idempotency, and health checks.	Protocol messages can be retried, replayed, quarantined, or acknowledged with explicit semantics.
Deployment	Broker serving, CLI diagnostics, SDK generation, compatibility matrix, local dev, and native-service control plane.	Protocol capability becomes deployable infrastructure, not per-project scripts.

What customers buy.

UDB Plus is not just a technical runtime. It is a product packaging model that lets teams sell protocol readiness as a controlled project capability on top of the UDB broker foundation.

16

FOR HEALTHCARE SOFTWARE**One platform for HL7v2 and FHIR coexistence.**

- Connect legacy feeds and modern APIs without two unrelated integration stacks.
- Preserve message-level evidence while projecting into project-owned models.
- Offer SDKs and data APIs to product teams instead of raw interface-engine scripts.

FOR INDUSTRIAL AND IOT SOFTWARE**One platform for MQTT and OPC UA data paths.**

- Model devices, topics, nodes, commands, alarms, and telemetry through one broker.
- Route real-time data into storage, analytics, dashboards, alerts, and APIs.
- Apply identity, authorization, and audit to command and control operations.

Faster onboarding

Projects opt into a protocol pack instead of writing a full protocol integration subsystem.

Lower risk

Versioned dictionaries, validation reports, provenance, and deterministic builds reduce hidden integration behavior.

Better SDK story

Developers use the same SDK posture for normal data and protocol-specific workflows.

Enterprise fit

Security, tenant isolation, audit, health, migrations, and eventing are broker-owned rather than project-owned.

Positioning statement.

UDB is the core broker. UDB Plus is the protocol extension layer. Together they let a product declare: this project uses UDB + HL7v2, UDB + FHIR, UDB + MQTT, UDB + OPC UA, or any combination needed by the customer.

TECHNICAL BUYER

Protocol behavior becomes inspectable infrastructure.

Parser rules, dictionaries, profiles, transports, storage, and SDKs can be reviewed, tested, and upgraded through the same platform lifecycle.

PRODUCT BUYER

Protocol support becomes a sellable project capability.

Teams can package domain readiness without fragmenting the core platform or building one-off integration engines.

OPERATIONS BUYER

Protocol traffic becomes governable.

Tenant/project isolation, health, audit, DLQ, replay, provenance, and evidence reports apply to protocol flows.

Recommended go-to-market language: "UDB Core powers the broker. UDB Plus adds protocol packs. Choose the packs your project needs: healthcare messaging, FHIR APIs, IoT telemetry, industrial node models, and future protocol domains."

PACKAGE LABEL	CUSTOMER-FACING DESCRIPTION
UDB Core	Schema-driven multi-backend data broker with native control plane, SDKs, migrations, events, storage, identity, authorization, and audit.
UDB Plus	Protocol-pack runtime for regulated and operational integrations that need parse, validate, transform, transport, persist, acknowledge, and audit features.
UDB + HL7v2 / FHIR / MQTT / OPC UA	Project-selectable protocol capabilities delivered on top of the same UDB broker, SDK, security, and operations model.